Project Report: CivicVoice

Team Number: 13
Team Members: Harsh Raj (2025202005), Chaitanya Prabhu (2025201100),
Deepti Koranga (2025204033), Prakash Bhabad (2025204016), Lalithamsh (2025202039)
Code Repository: https://github.com/DeeptiKoranga/CivicVoice.git
Presentation:
Demo Video: https://youtu.be/XP3ObswnLxI

About Project

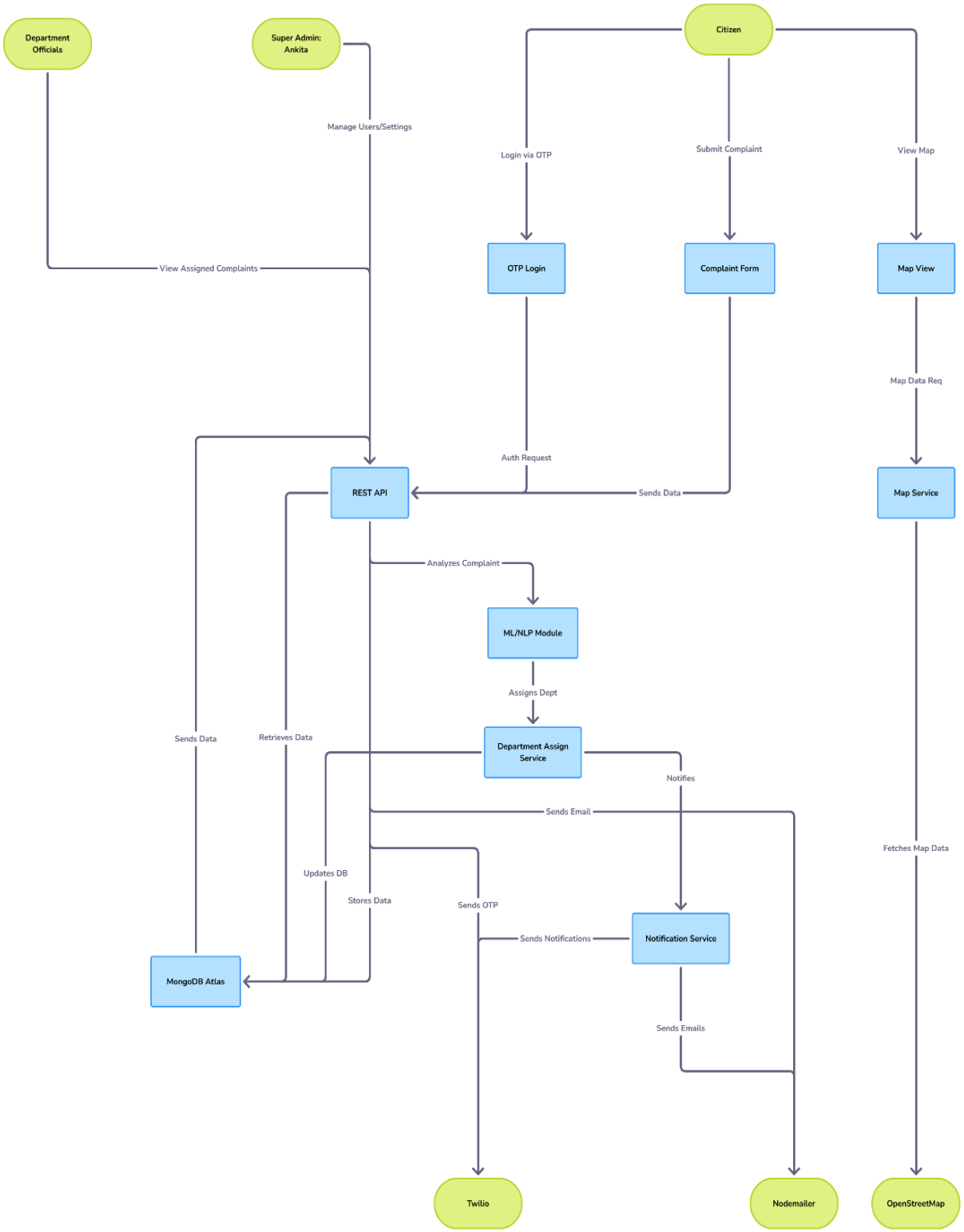
Problem Statement

Rapid urbanization has led to increasing civic issues such as potholes, waste mismanagement, water supply disruptions, and sanitation failures. Existing reporting mechanisms are slow, non-transparent, and often inaccessible, resulting in citizens feeling unheard and authorities not receiving accurate ground-level inputs.

Proposed Solution

CivicVoice is a web-based public grievance reporting and city-response platform that bridges the communication gap between citizens and municipal authorities. It features a **publicly visible, interactive city map** where all complaints can be viewed, along with a detailed admin dashboard for complaint verification, department routing, and resolution tracking. The system uses OTP-based citizen login, automated NLP summarization, chatbot support, and real-time SMS notifications.

Solution Diagram



External Users

- Citizen – initiates login and complaint submission.
- Super Admin (Ankita) – manages verification, assignment, and system settings.
- Department Official – views assigned complaints and uploads resolution proof.

These users communicate with the system through the frontend UI.

Frontend Layer (React Application)

Feature	Description
OTP Login	Citizen enters mobile number. OTP request is sent to backend via REST API. Twilio delivers OTP SMS.
Complaint Form	User enters complaint, uploads images/videos, and pins location. Data is sent to the backend for processing.
Map View	Retrieves map data through the Map Service. Uses OpenStreetMap tiles fetched via backend service.

Backend Layer (Node.js / Express)

Module	Key Functions
REST API	Login requests, Complaint submissions, Map data requests, Admin verifications, Department actions
NLP / ML Module	Analyzes and summarizes complaint text, Returns a concise summary for the admin dashboard
Department Assign Service	Suggests ideal department (e.g., Roads, Water, Sanitation), Sends assignment email through Nodemailer, Updates assignment status in MongoDB
Map Service	Fetches location and tile data from OpenStreetMap, Provides updated map overlays for the frontend
Notification Service	Sends OTP via Twilio, Sends complaint update SMS, Sends email notifications (department alerts, admin messages)

Database Layer (MongoDB Atlas)

Stores all persistent data:

- Citizen details
- Complaints + media URLs
- Status updates
- Admin actions
- Department assignments
- Map coordinates
- Summaries produced by NLP

Integrations (External Services)

Component	Function
Twilio	• Delivers OTPs • Sends SMS updates at verification & resolution
Nodemailer	• Sends email notifications to departments • Sends admin alerts
OpenStreetMap	• Provides map tiles & geolocation data • Supports complaint placement on the public map

Complaint Flow Pipeline

The flow moves like this:

1. Citizen submits complaint → REST API
2. REST API → NLP Module (summary generated)
3. Summary + details → Admin Dashboard
4. Admin verifies → Department Assign Service
5. Department notified → Email + Dashboard
6. Department uploads proof → REST API
7. Database updated → MongoDB
8. Resolved status → Citizen notified via SMS
9. Updated map data provided through Map Service

This pipeline visually shows Citizen -> Admin -> Department -> Admin -> Citizen feedback loop.

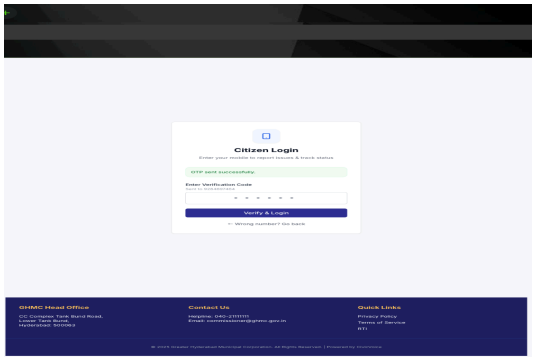
Personas and User Journey

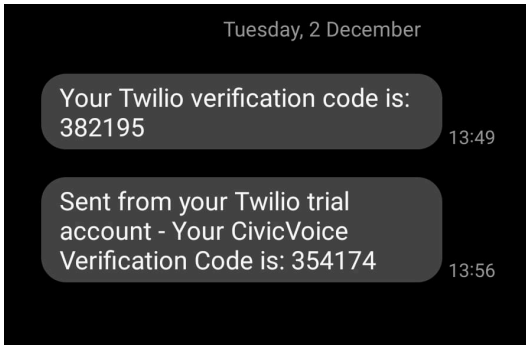
Stakeholders

1. **Citizen** – raises complaints, tracks resolution, views public map.
2. **Super Admin (Ankita)** – verifies complaints, assigns departments.
3. **Department Officials** – resolve the issue and upload proof.

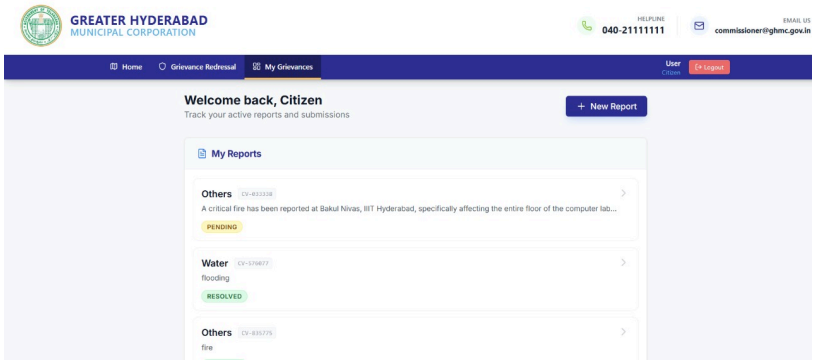
A. Citizen Journey

1. **Login** – Citizens authenticate using mobile number + OTP.

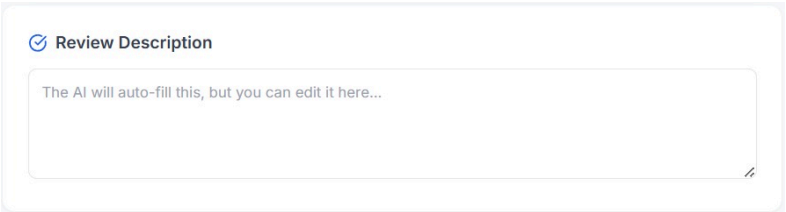




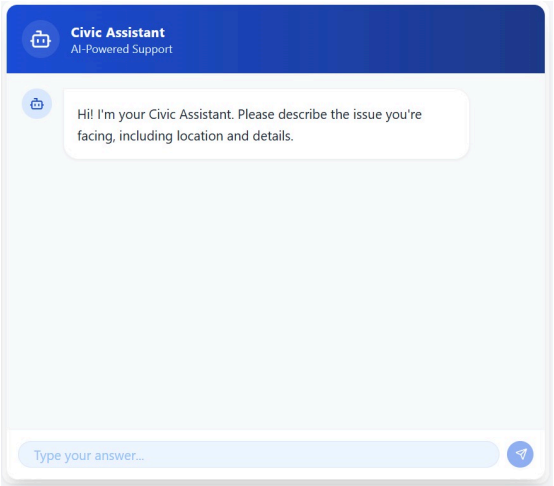
2. **Report** – Submits complaint via text, image, video; location pinned on map.



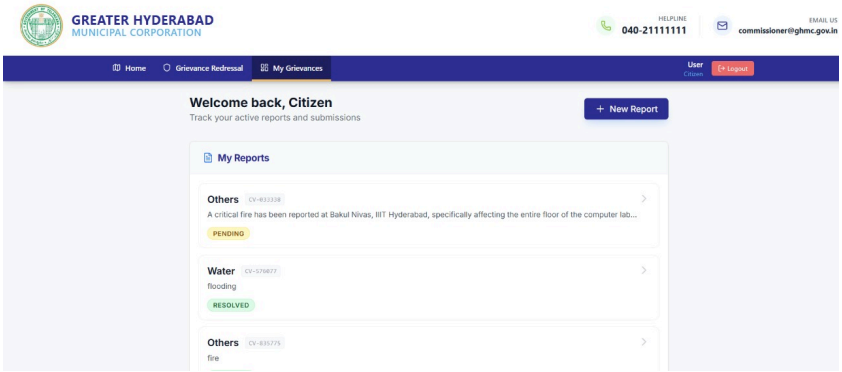
3. **NLP Summarization** – Complaint is auto-summarized using NLP.



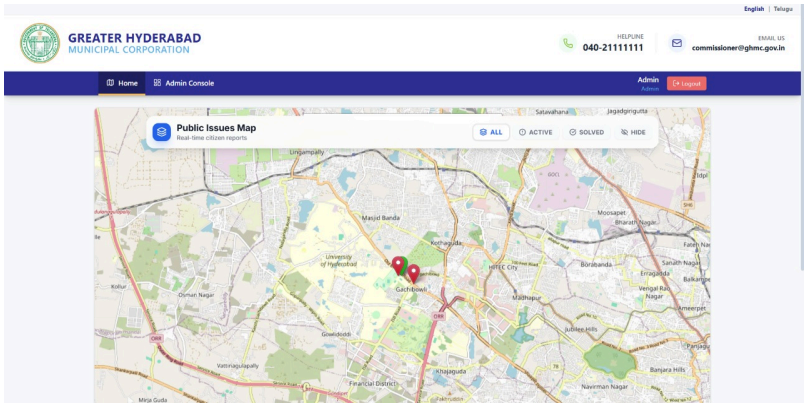
4. **Chatbot Support** – A decision-tree-based chatbot helps users classify or refine complaints.



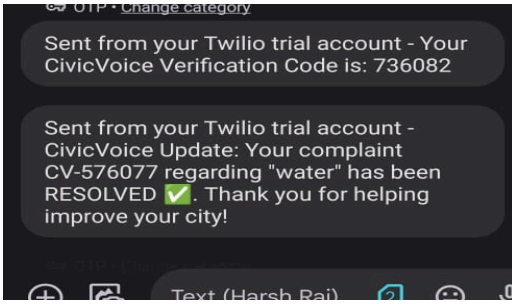
5. **Track** – Citizen views complaint status: Pending -> Verified -> Resolved.



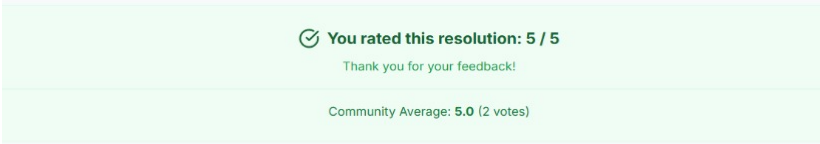
6. **Public Visibility** – Complaints appear on public map for all citizens.



7. **Notifications** – Receives SMS updates at verification and resolution.

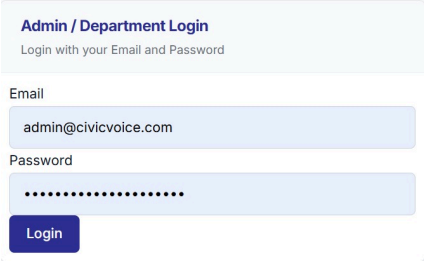


8. **Feedback** – Provides star-rating after resolution.

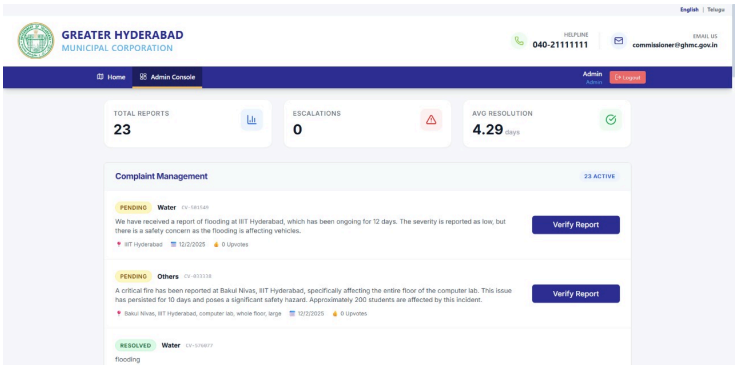


B. Admin Journey (Ankita)

1. **Login** using secured credentials.

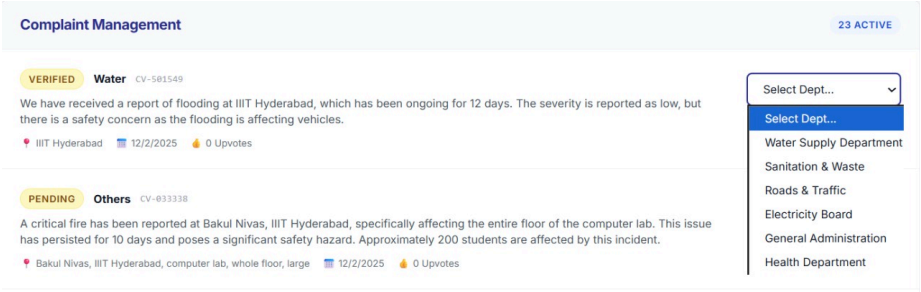


2. **Dashboard View** – Total complaints, active issues, resolved issues, average resolution time.

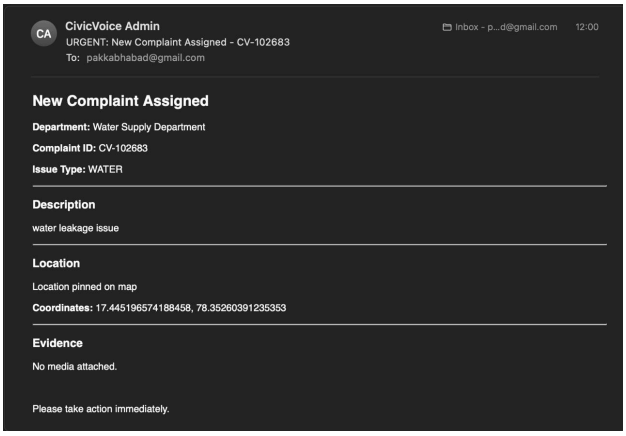


3. **Complaint Verification** – Reviews citizen complaints along with NLP-generated summaries.

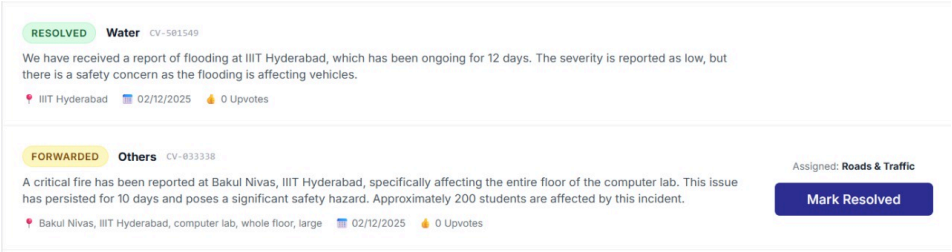
4. **Department Assignment** – Auto-suggested department populated; admin finalizes selection.



5. **Notifications** – Department receives automated email; citizen gets status update.



6. **Close Issue** – Admin marks complaint as resolved and final SMS is triggered.



Software / Hardware

Technology Stack

Category	Technologies
Frontend	React.js, Vite, Tailwind CSS, Framer Motion, Leaflet + React-Leaflet (Interactive Maps), Axios
Backend	Node.js, Express.js
Database	MongoDB Atlas
Authentication	JWT
External Services	Twilio (OTP + SMS notifications), Nodemailer (Email notifications), OpenStreetMap Tiles & Geocoding, Cron Jobs (Auto-escalation after 7 days)

Hosting Instructions (Local Setup)

Step 1: Clone and Install

```
git clone <repository_url>
cd civicvoice

# Install Backend
cd backend
npm install

# Install Frontend
cd ../frontend
npm install
```

Step 2: Environment Configuration

Create a `.env` file in the backend:

```
PORT=5001
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/civicvoice
JWT_SECRET=your_secret_key
TWILIO_SID=your_sid
TWILIO_AUTH_TOKEN=your_token
TWILIO_PHONE=your_twilio_number
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
```

Step 3: Run the Application

- Start Backend:
  `node server.js`
  (Runs on Port 5001)

- Start Frontend:
  `npm run dev`
  (Runs on Port 5173)

Access the app at: **http://localhost:5173**

Contribution

Team Member Role	Key Responsibilities/Tasks
Harsh Raj	• Designed MERN system architecture and database schema (Users, Complaints, Departments). • Implemented secure REST APIs for complaint handling and routing. • Developed OTP login system and authentication flow. • Built automated escalation using cron jobs.
Chaitanya Prabhu	• Integrated Twilio for OTP + SMS notifications. • Added Nodemailer email service for department communications. • Implemented Map service backend (OpenStreetMap tiles, geocoding, data fetch). • Developed NLP module for complaint summarization.
Lalithamsh	• Designed high-fidelity screens for citizen and admin workflows. • Built modern UI layouts applying Tailwind design principles. • Created user flow diagrams, personas, and interaction patterns. • Designed map UI, complaint form interface, chatbot interface.
Prakash Bhabad	• Built responsive React interface using Tailwind + Framer Motion. • Developed complaint form, filters, NLP summary display, and map view • Connected frontend to backend REST APIs using Axios. • Implemented OTP login screens and My Reports page.

Deepti Koranga	• Developed admin dashboard: analytics view, verification screens, assignment panel, and resolution workflow. • Implemented department suggestions and proof upload components. • Integrated notification indicators and real-time updates. • Optimized frontend state management and data flows.

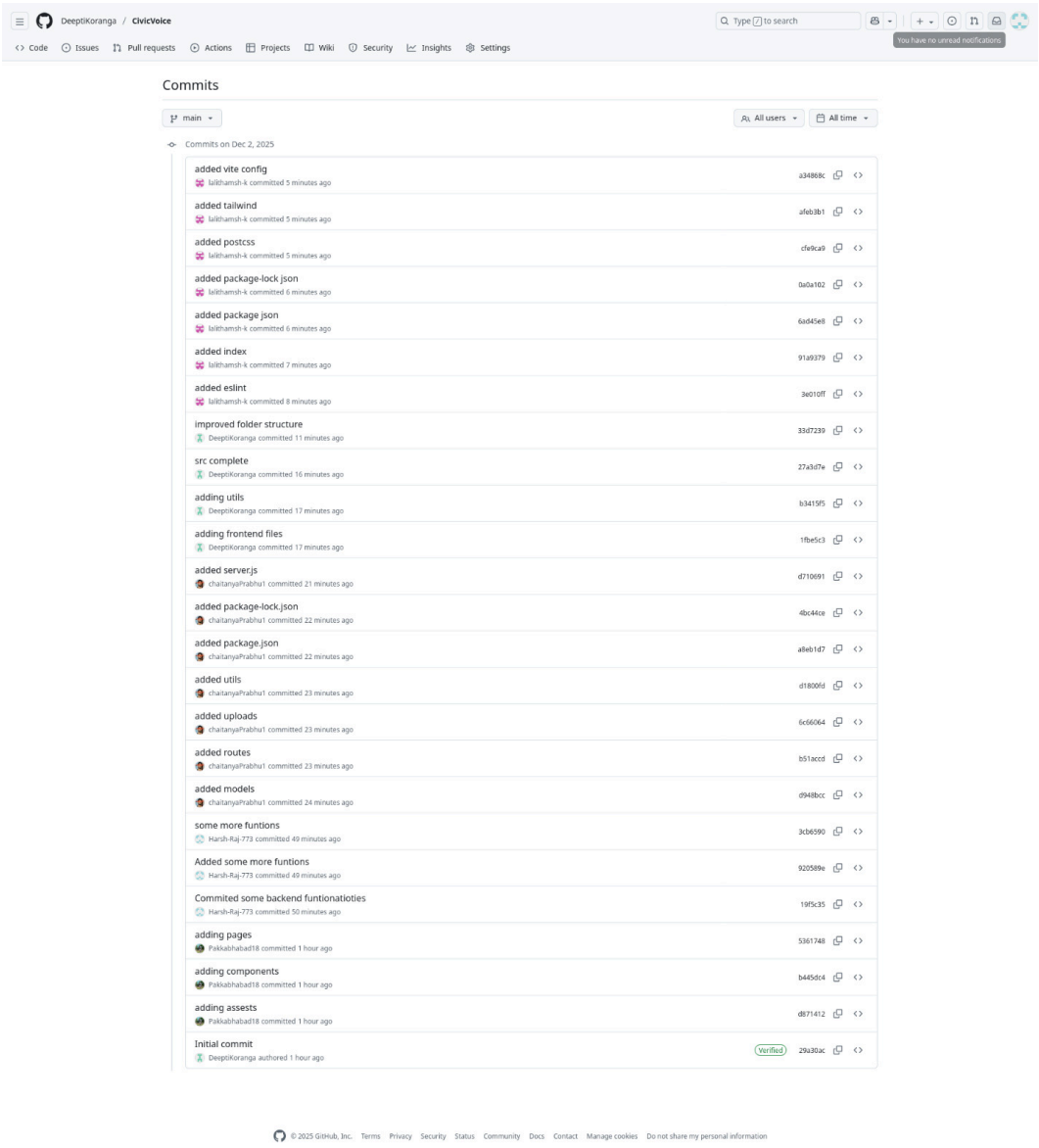


Fig: Git Commit History

References

- React Documentation — https://react.dev
- Leaflet Maps API — https://leafletjs.com
- Twilio Documentation — https://www.twilio.com/docs
- MongoDB Atlas — https://www.mongodb.com/atlas
- Tailwind CSS — https://tailwindcss.com